NETMAP

A Network Discovery and Security Dashboard Using Nmap, Python and Wireshark

Course Project Report | Assessment 9

BCSE308P - Course Project

Yashes Kartik | Register Number: 24BYB0071

Faculty Guide: Dr Navaneethan

Department of Cybersecurity and Intelligent Systems

School of Computer Science and Engineering (SCORE)

Vellore Institute of Technology, Vellore

Submission Date: 23 October 2026

DECLARATION

I, Yashes Kartik (Register Number 24BYB0071), declare that the course project report titled "NetMap: A Network Discovery and Security Dashboard Using Nmap, Python and Wireshark" is submitted in partial fulfilment of the requirements of BCSE308P - Course Project at Vellore Institute of Technology, Vellore.

The report describes the NetMap application, implementation and observations recorded in the saved scan artefacts. Results are presented with their scope and limitations. The two Nmap XML files and corresponding text files are the primary records for scan findings. The packet capture is described separately because its visible and filtered traffic does not corroborate the target scan.

External tools and documentation are identified in the references. Any further institutional declaration regarding originality or AI assistance should be completed according to current course instructions.

Name: Yashes Kartik | Register Number: 24BYB0071

Date: 23 October 2026

CERTIFICATE

This is to certify that the course project report titled "NetMap: A Network Discovery and Security Dashboard Using Nmap, Python and Wireshark" is submitted by Yashes Kartik (Register Number 24BYB0071) in partial fulfilment of the requirements for BCSE308P - Course Project at Vellore Institute of Technology, Vellore.

The work documented here covers a Python and Streamlit application that interprets saved Nmap XML, presents host and port records, compares two scan snapshots and provides CSV/HTML report outputs. The network observations are limited to the saved artefacts described in the findings chapter.

Faculty Guide: Dr Navaneethan

ACKNOWLEDGEMENT

I thank Dr Navaneethan for guidance and feedback during development of this course project. I also acknowledge the Department of Cybersecurity and Intelligent Systems and the School of Computer Science and Engineering at VIT for the academic setting in which the work was completed.

The project uses Nmap for network discovery and service fingerprinting, Python for parsing and reporting, Streamlit for the local dashboard, and Wireshark for independent packet inspection. Their official documentation informed the implementation and interpretation of evidence.

I am grateful to everyone who helped review the application and its presentation. This report distinguishes verified scan records from the limitations in packet-capture evidence so the conclusions remain reproducible and appropriately scoped.

EXECUTIVE SUMMARY

NetMap is a local dashboard for reviewing saved Nmap XML scans. It converts structured scan output into host and port tables, summary metrics, a port-state chart, scan reports and a before/after comparison. The application does not launch scans from the dashboard; it accepts a saved XML file selected from the project or uploaded by the user. This separation keeps acquisition and review explicit.

The observed data came from two Nmap 7.80 TCP SYN service-detection scans of private address 172.17.104.158, each covering the 100 most common TCP ports. Both XML files report one host up, four open TCP ports (135, 139, 445 and 8080), and 96 closed ports. The scans began at 22:17:39 and 22:39:55 on 22 October 2026. NetMap reports no differences between open endpoint and service labels in these snapshots.

The project's 15 automated tests passed; Python compilation and dependency checks also passed in the recorded verification. The saved PCAPNG has 94 packets, but the supplied Wireshark screenshot shows loopback traffic and target-specific inspection recorded no scan SYN frames for the four ports. Therefore the capture is not independent confirmation of the Nmap results. Port 8080's product string is a fingerprint, not proof that Oracle software is installed.

CONTENTS, FIGURES AND ABBREVIATIONS

Declaration ii

Certificate iii

Acknowledgement iv

Executive Summary v

1. Introduction 1

2. Project Description and Goals 3

3. Technical Specification 5

4. Design Approach and Details 8

5. Schedule, Tasks and Milestones 12

6. Project Demonstration 13

7. Cost Analysis, Results and Discussion 21

8. Summary 24

9. References 25

Appendix A. Reproduction and verification 27

Figures: 4.1 NetMap architecture; 6.1 Scan selection; 6.2 Overview; 6.3 Host Discovery; 6.4 Ports & Services; 6.5 Comparison; 6.6 Exports; 6.7 Nmap CMD; 6.8 Wireshark.

Tables: 2.1 Goals; 3.1 Components; 4.1 Comparison rules; 5.1 Tasks; 7.1 Scan findings; 7.2 Verification.

Abbreviations: API, CSV, HTML, IP, PCAPNG, SYN, TCP, UDP, XML.

CHAPTER 1 INTRODUCTION

1.1 Background. A network inventory begins with identifying reachable hosts and services that respond on selected ports. Nmap performs active probing and can report host status, port state and service fingerprints. Its XML output is designed for programmatic processing and includes structured elements that are more stable for tools than terminal-formatted output [1, 2].

A scan result is a time-bounded observation. A port reported open means that the scan received a response Nmap classified as open under the selected scan method. It does not establish that the service is vulnerable, misconfigured or exposed along every network path. Context such as target, interface, timing, ports and firewall behaviour affects interpretation.

NetMap addresses the review step after acquisition. The user loads an existing Nmap XML file, and the application presents its contents consistently across overview metrics, host records, port/service records and exports. A second XML file can be compared against a baseline to surface changes without replacing original data.

PROBLEM STATEMENT, OBJECTIVES AND SCOPE

1.2 Problem statement. Raw scan output can be cumbersome to read when a user needs a concise inventory or must compare scan dates. A terminal table does not provide an interactive overview, consistent downloadable records, or a transparent comparison of endpoint identity and service labels.

The project was motivated by a need for a small coursework-scale tool demonstrating an end-to-end analysis chain: import structured evidence, normalize it, render it in a dashboard, compare snapshots and export results. A central design requirement is source consistency. Overview numbers, charts and records must come from the same selected XML file; synthetic fixtures must not be blended into real observations.

Wireshark offers an independent way to inspect captured packets, but a capture is useful only when it includes traffic relevant to the scan. NetMap therefore treats packet analysis as a separate evidence source and records when a capture cannot validate the scan.

1.3 Objectives. Parse valid Nmap XML; preserve host, port, service, scan-time and reason metadata; show a clear source label; compare open endpoints by IP, port and protocol; identify opened, no-longer-open and changed-service records; and export single-scan and comparison reports.

1.4 Scope. NetMap is a local review dashboard and parser. It does not initiate network probes through Streamlit, store a central scan database, perform vulnerability scoring, parse PCAPNG inside the application, or assert that a fingerprint is ground truth. The CLI module has separate guarded scan functions, but the dashboard workflow described here loads saved XML only.

The demonstrated scan scope is one private IPv4 address and the top 100 TCP ports. Conclusions do not generalize to a subnet, UDP, IPv6, another interface or another time. The contribution is an integrated review and comparison workflow with explicit handling of data provenance and empty/error cases.

CHAPTER 2 PROJECT DESCRIPTION AND GOALS

2.1 Product description. NetMap is a Python project whose Streamlit interface reads saved Nmap XML. A user selects an XML file from the project scans directory or uploads one. After validation, the app stores that parsed scan in session state and exposes Overview, Host Discovery, Ports & Services, Comparison, and Reports & Exports views.

A separate comparison workflow asks for Baseline scan and After scan inputs. Each input may select a project XML file or upload an independent XML file. The comparator matches endpoints by (IP address, port number, protocol) and reports differences only in open endpoints or service labels. It retains timestamps and filenames for traceability.

The command-line module also parses XML and writes CSV/HTML reports. Existing scan artefacts are kept under scans/; report outputs are under outputs/; automated tests and clearly labelled fixture XML are separate. The dashboard does not load synthetic fixtures in a normal user session.

Table 2.1. Goals and measurable indicators

Goal	Indicator	Observed / verified status
Consistent real-scan overview	Metrics, chart and records share one XML source	Verified in UI evidence and AppTest coverage
Clear provenance	Filename and Nmap timestamp visible	Real-source banner and captions
Useful comparison	Three change categories and endpoint key	15-test suite includes key comparison case
Portable reports	CSV/HTML for scans and comparisons	Files present; contents inspected locally
Graceful validation	Reject invalid XML; empty states	Covered by automated tests
No fabricated demo data	Production app starts at load gate	Verified in tests and README

USERS, USE CASES AND REQUIREMENTS

2.2 Intended users. The primary user is a student or analyst reviewing an authorized scan already collected with Nmap. The interface assumes the user can identify the correct baseline, later scan and target scope. It is not an access-control system and cannot determine whether the user is authorized to inspect a target.

2.3 Main use case. The user opens NetMap, selects a saved XML scan, checks source and timestamp, reviews hosts and ports, and optionally downloads CSV/HTML. To compare, the user supplies earlier and later XML, reviews three change categories and exports the comparison. Invalid input produces an error and does not replace a previously loaded valid result with stale or partial data.

Non-functional goals include local execution, understandable errors, safe handling of uploaded filenames, deterministic output fields, readable tables, and preservation of the existing light design. The dashboard is intended for one user session, not concurrent multi-tenant operation.

CHAPTER 3 TECHNICAL SPECIFICATION

3.1 Runtime and dependencies. The application is implemented in Python. The parser uses standard-library ElementTree for XML structure and csv for delimited output. Streamlit provides the local web interface; pandas prepares tabular data; Altair renders the port-state chart. Nmap produces XML input, and Wireshark is used separately to inspect packet captures.

The recorded verification used the project's Windows Python launcher and dependency environment. The test log reports Python compilation success, all 15 unittest cases passing and pip check reporting no broken requirements. These checks validate code/import consistency and fixture-driven behaviour; they do not prove an active scan was rerun in that verification session.

Nmap XML includes scan metadata, hosts, addresses, port states and service attributes. XML lets a general parser read structured data without scraping terminal text [1, 2]. ElementTree provides APIs for parsing XML documents and finding child elements [3].

SYSTEM COMPONENTS AND RESPONSIBILITIES

Table 3.1. System components and responsibilities

Component	Input	Responsibility	Output
Nmap 7.80	Authorized target and options	Host discovery, TCP probing and fingerprinting	XML and text records
netmap.py parser	Nmap XML file	Normalize time, hosts, addresses and ports	Host and port rows
Comparator	Two parsed scans	Compare open endpoint keys and service labels	Change rows or empty result
Streamlit dashboard	One scan or two comparison scans	Source-consistent metrics, charts, tables and controls	Interactive local review
Report writers	Normalized records	CSV and escaped HTML output	Portable reports
Wireshark	PCAPNG capture	Independent packet inspection	Analyst observations

DATA MODEL, SOURCE FIELDS AND DEPLOYMENT

3.2 Scan metadata. The parser obtains a human-readable start string from the Nmap root when present and can fall back to a numeric timestamp. The root element must be `nmaprun`. Parse failures or a non-Nmap root become a helpful ValueError.

Each host record preserves scan time, a selected address, address type, observed addresses, host state, state reason and the first hostname when present. Each port record preserves scan time, host address/status, port number, protocol, state, service name, product, version, detection method, confidence and state reason. Missing optional XML values receive safe defaults rather than terminating the parse.

The comparator operates over open records, not every closed or filtered record. It answers a focused question: which endpoints are newly open, no longer observed open, or still open with a changed service label? A missing endpoint is "no longer open," not "closed," because a later scan could omit it for reasons other than a confirmed closed state.

3.3 Deployment model. The dashboard runs locally with Streamlit and reads files selected from the scans directory or uploaded by the user. It does not require a server-side database. Its workflow uses session state for the active scan; changing the scan clears the selection and returns to the load screen.

The saved scans use Nmap 7.80 and were initiated with `-Pn -sV --top-ports 100 -T3 -oX ... -oN ...` against 172.17.104.158. `-Pn` skips host discovery; `-sV` requests service/version detection; the top-ports option limits the port set. Interpretation must retain these options because different scan profiles can yield different observations [4-6].

NetMap is an educational local tool. It does not claim high availability, authentication, role-based access, encryption at rest or centralized retention. These omissions suit coursework use but would need review before organizational deployment.

ARCHITECTURE AND DATA FLOW

4.1 Method and architecture. The design followed an evidence-first workflow: inspect the existing app and artefacts, retain the light dashboard, normalize XML records, preserve source identity, compare stable endpoint keys and verify edge cases with fixtures and Streamlit AppTest. Nmap XML is the boundary between scan acquisition and review; the dashboard reads saved XML but does not launch Nmap.

The data path is scan invocation -> XML/text outputs -> validation and parsing -> normalized host/port rows -> overview, tables, charts, comparison -> CSV/HTML. The packet path is separate: capture interface -> PCAPNG -> Wireshark inspection. Conclusions combining both paths require the capture to match target, ports and time window.

A structured XML boundary avoids scraping terminal text [1]. Reviewing saved files also permits analysis without generating new traffic.

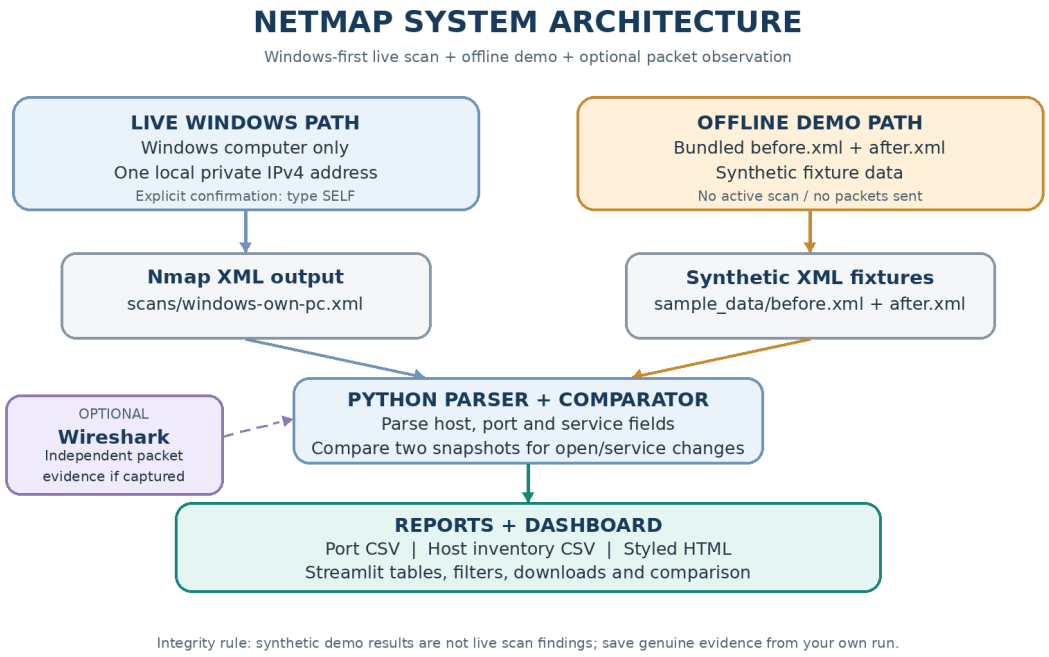


Fig. 4.1. NetMap architecture and separation of scan review from packet capture.

PARSER DESIGN AND VALIDATION

4.3 XML parsing. `parse_scan` receives a path and asks ElementTree to parse the document. File and XML errors are caught and re-raised as a user-facing ValueError. It checks that the root is `nmaprun`, then reads scan time, hosts, addresses, host status, port state and optional service data. Hosts without a ports element remain valid host records; ports without service metadata remain valid port records.

The UI upload path writes uploaded bytes to a short-lived temporary file with a fixed safe name instead of trusting the user-supplied filename as a destination. The same parser handles the upload. Files from scans/ are validated before their records enter session state.

An empty but valid scan differs from malformed XML. The former can be displayed with a no-host/no-port/no-open-port state. The latter is rejected with an explanation. This distinction lets a genuine empty observation be reported without treating it as software failure.

```
root = ET.parse(file_path).getroot()
if root.tag != "nmaprun":
    raise ValueError("Not Nmap XML")
for host in root.findall("host"):
    # normalize addresses, status, ports and service metadata
```

COMPARISON ALGORITHM AND RULES

4.4 Endpoint identity. Each open port is keyed by host IP, port number and transport protocol. Including protocol matters because TCP/53 and UDP/53 are different endpoints. For matching keys, NetMap compares a service label built from service name, product and version.

Set differences between old and new endpoint maps determine opened and no-longer-open records. Their intersection is checked for changed labels. Results are sorted for deterministic output. Each change row carries baseline and later filenames and timestamps.

A product-text change is a changed scanner fingerprint, not necessarily a software upgrade. A no-longer-open row is not automatically a closed-port finding. The later XML state must be examined before assigning a more specific explanation.

Controlled fixture XML tests changed endpoint cases. Those tests verify algorithm behaviour, not actual network changes. The saved real scan pair is an observational comparison and produces an empty change set.

Table 4.1. Rules for categorizing scan differences

Condition	Result	Interpretation
Key exists after, absent before	opened	Newly observed open
Key exists before, absent after	no_longer_open	Not observed open later
Key open in both; label differs	service_changed	Fingerprint label changed
No qualifying differences	Empty change set	No differences detected

INTERFACE, EXPORTS AND TRADE-OFFS

4.5 Interface. A fresh session presents a scan-selection gate with no metrics or sample rows. After a valid scan loads, the same session data feeds Overview, Host Discovery, Ports & Services and Reports & Exports. Change scan removes the active selection. Comparison uses separate baseline and after widgets with distinct keys.

CSV suits tabular review and spreadsheet import. HTML provides a standalone readable table and includes source/time metadata. The HTML writer escapes title, subtitle and cell values. Output fields remain explicit and ordered. Comparison exports preserve both scan identities even when the change table is empty.

Trade-offs include a lightweight local session model instead of a database, a focused open-endpoint comparator instead of broad state analytics, and saved-XML review instead of a scan scheduler. These choices simplify the safety model but require users to manage scans and select equivalent scopes themselves.

CHAPTER 5 SCHEDULE, TASKS AND MILESTONES

5.1 Work sequence. The project proceeded through report-format review; repository and evidence inspection; stable scan loading and provenance; the two-input comparison path; CSV/HTML output; edge-case regression tests; and final report assembly. XML parsing and source tracking precede dashboard metrics; comparison depends on normalized records; findings depend on saved scan artefacts.

The sequence reflects dependencies rather than invented hour estimates. Report evidence was assembled after the scan files and existing screenshots were inspected. Existing Nmap outputs were preserved; no new network scan was needed to create this report.

Table 5.1. Work packages and deliverables

Work package	Dependency	Deliverable / evidence
Repository and QP review	Project and supplied guide	Scope, chapter outline and formatting rules
XML and provenance workflow	Parser and saved scans	Real scan loading and source/time display
Comparison and exports	Normalized host/port records	Baseline/After flow and CSV/HTML
Regression verification	Stable behaviour	15 tests, py_compile, pip check
Evidence review	Existing screenshots/artefacts	UI, command and capture figures
Assessment report	Verified evidence	Synchronized DOCX/PDF

SCAN SELECTION SCREEN

6.1 Demonstration scope. These figures document application workflow and supplied evidence. Dashboard screenshots show a selected real XML file. The command screenshot is a view of saved Nmap text output; it demonstrates the recorded command/result, not a newly executed scan. The Wireshark screenshot shows the capture opened in the GUI, but its visible loopback packets do not validate the target's four open ports.

Screenshots are evidence of application states and artefacts. They do not replace XML, tests or packet-level analysis. Each caption states what an image establishes and its limits.

6.2 Loading a real scan. The initial screen requires selection of a valid scan from the project scans directory or an upload. The application does not populate the page with demo metrics before a successful load. This gate creates a clear start state and reduces the risk that fixtures will be mistaken for measurements.

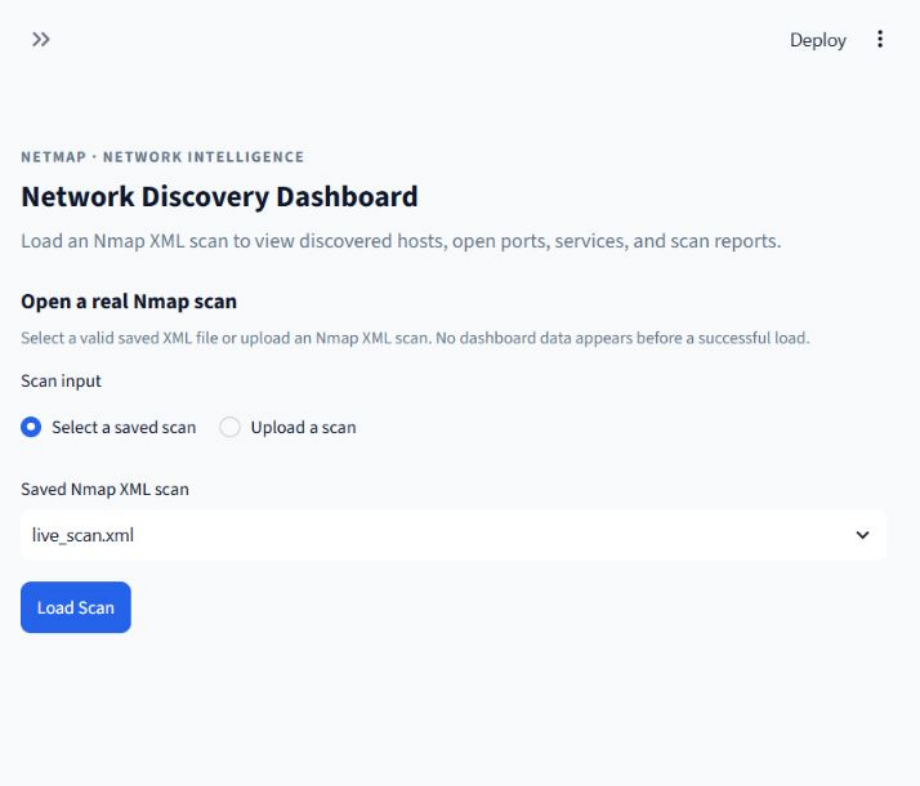


Fig. 6.1. Real Nmap XML selection screen; the dashboard waits for a successful load.

REAL-SCAN OVERVIEW

6.3 Overview. The Overview presents metrics, state chart, host/port records and scan metadata from the loaded XML. It labels source and scan time. The documented scan reports one host and four open ports among the 100 selected TCP ports. The chart and table are tied to that XML rather than to a synthetic baseline.

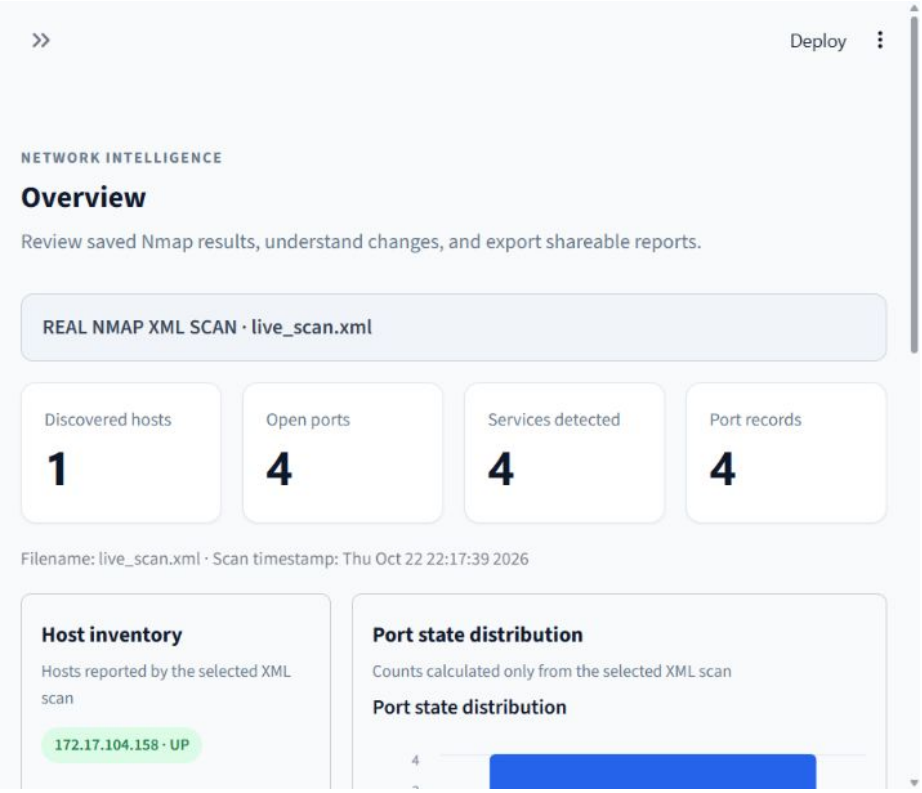


Fig. 6.2. Overview populated from a real saved Nmap XML scan.

HOST DISCOVERY VIEW

6.4 Host records. The host view presents the discovered host and status. For the saved scan, the target is 172.17.104.158 and Nmap records it as up. A single observed host is consistent with a one-address scope; it is not evidence of a broader inventory.

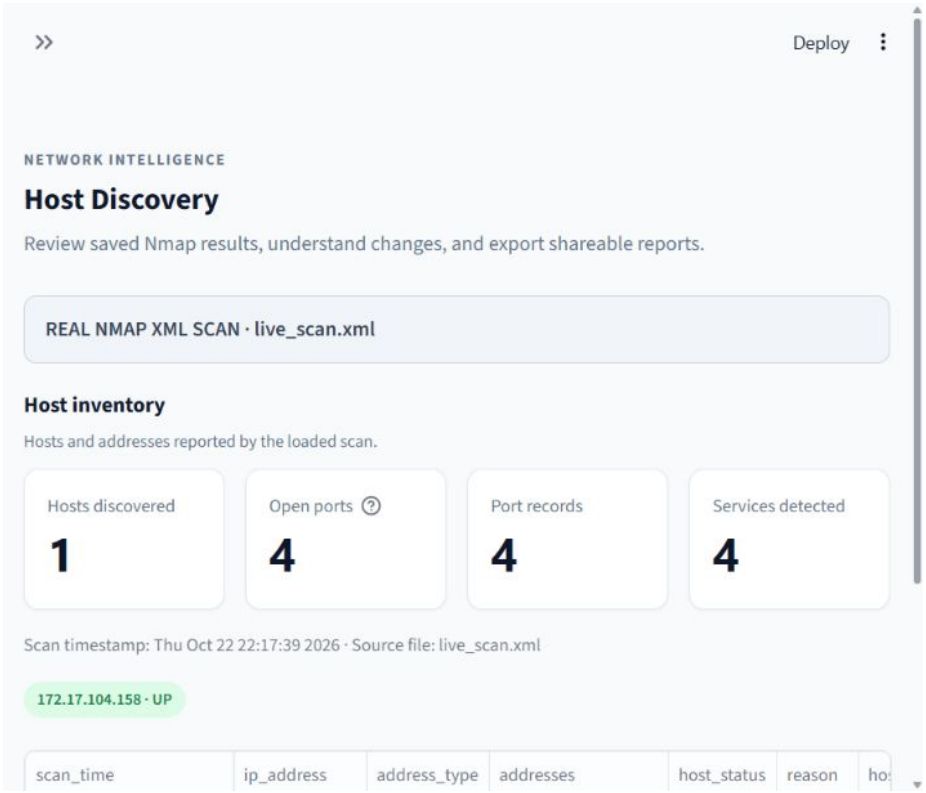
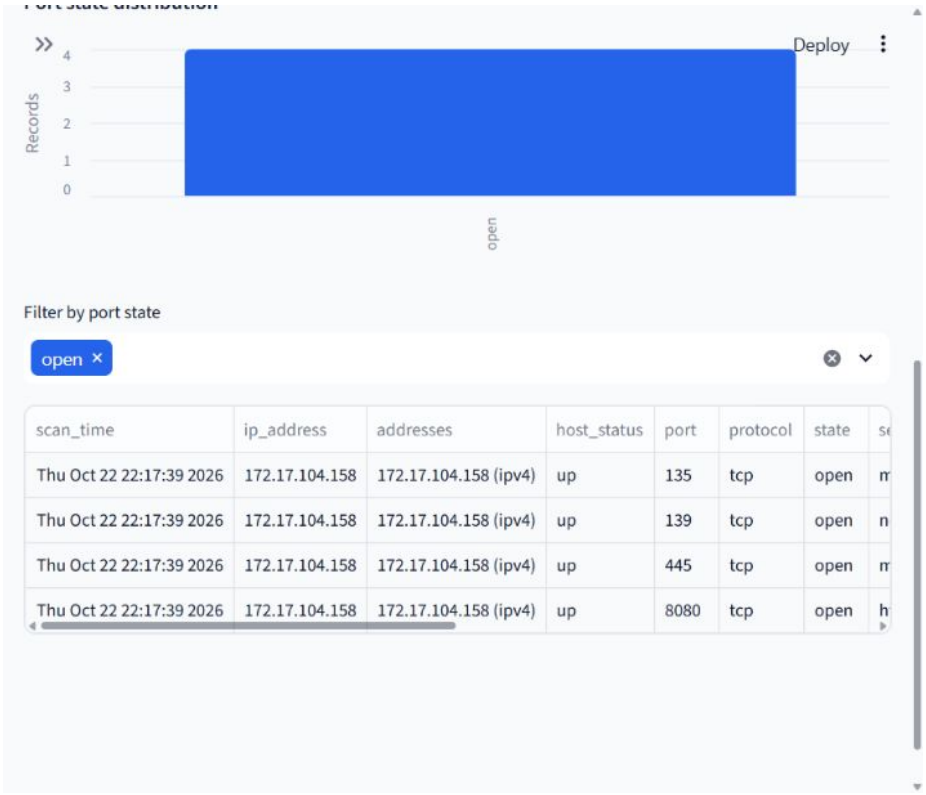


Fig. 6.3. Host Discovery view using the same active real scan.

PORTS AND SERVICES VIEW

6.5 Port/service records. The view exposes four open endpoints and associated labels: TCP/135 msrpc, TCP/139 netbios-ssn, TCP/445 microsoft-ds and TCP/8080 http. Product/version fields appear only when present in XML. This preserves the distinction between Nmap's service name and its fingerprint product string.



scan_time	ip_address	addresses	host_status	port	protocol	state	s
Thu Oct 22 22:17:39 2026	172.17.104.158	172.17.104.158 (ipv4)	up	135	tcp	open	m
Thu Oct 22 22:17:39 2026	172.17.104.158	172.17.104.158 (ipv4)	up	139	tcp	open	n
Thu Oct 22 22:17:39 2026	172.17.104.158	172.17.104.158 (ipv4)	up	445	tcp	open	m
Thu Oct 22 22:17:39 2026	172.17.104.158	172.17.104.158 (ipv4)	up	8080	tcp	open	h

Fig. 6.4. Ports & Services view; rows derive from the selected XML.

BASELINE AND AFTER COMPARISON

6.6 Comparison. The baseline is `live_scan.xml` and the later file is `live_scan_after.xml`. Both use target 172.17.104.158, Nmap 7.80, TCP SYN scan method and top-100 TCP scope. The comparison shows zero newly open endpoints, zero no-longer-open endpoints and zero changed service labels. Its empty-state message is "No differences detected."

This means the saved snapshots have the same open endpoint keys and service labels. It does not establish that all settings or packets between scans were identical.

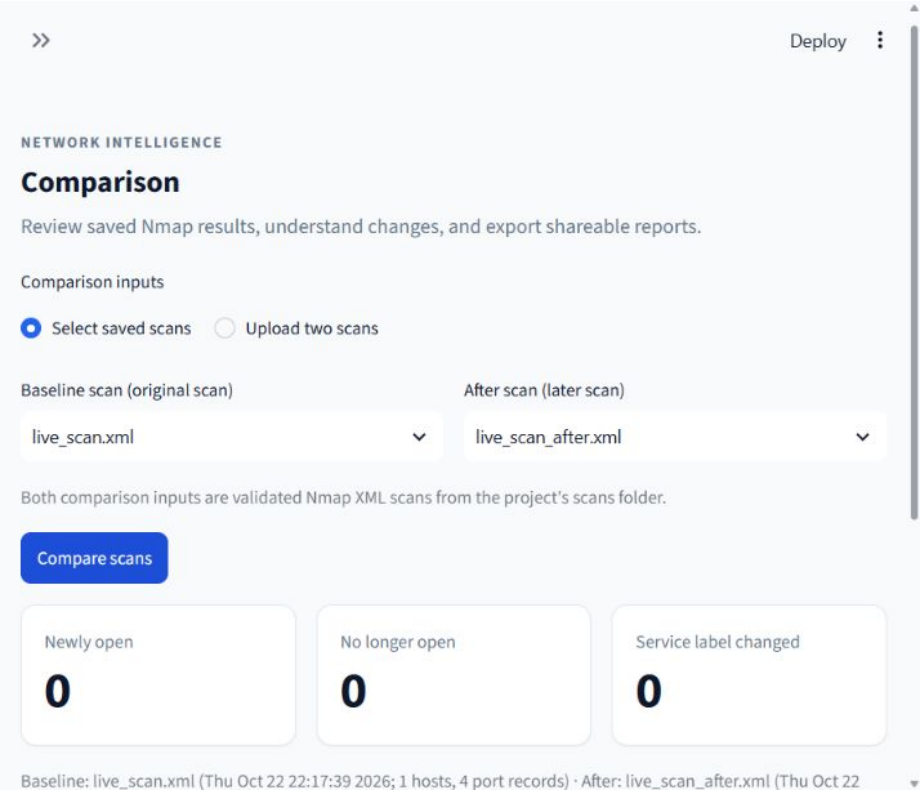


Fig. 6.5. Baseline/After comparison view showing no detected differences.

REPORTS AND EXPORT CONTROLS

6.7 Reports. The dashboard exposes port CSV, host CSV and styled HTML for the active scan, plus CSV and HTML for a selected comparison. The repository contains CLI-generated outputs from the saved XML. The single-scan CSV has four port rows; the host CSV has one host row. The comparison CSV has a header and no change records, while HTML states that no differences were detected.

The screenshot documents the export controls. Existing files in outputs/ were inspected locally; this report does not claim that a browser download was opened during report preparation.

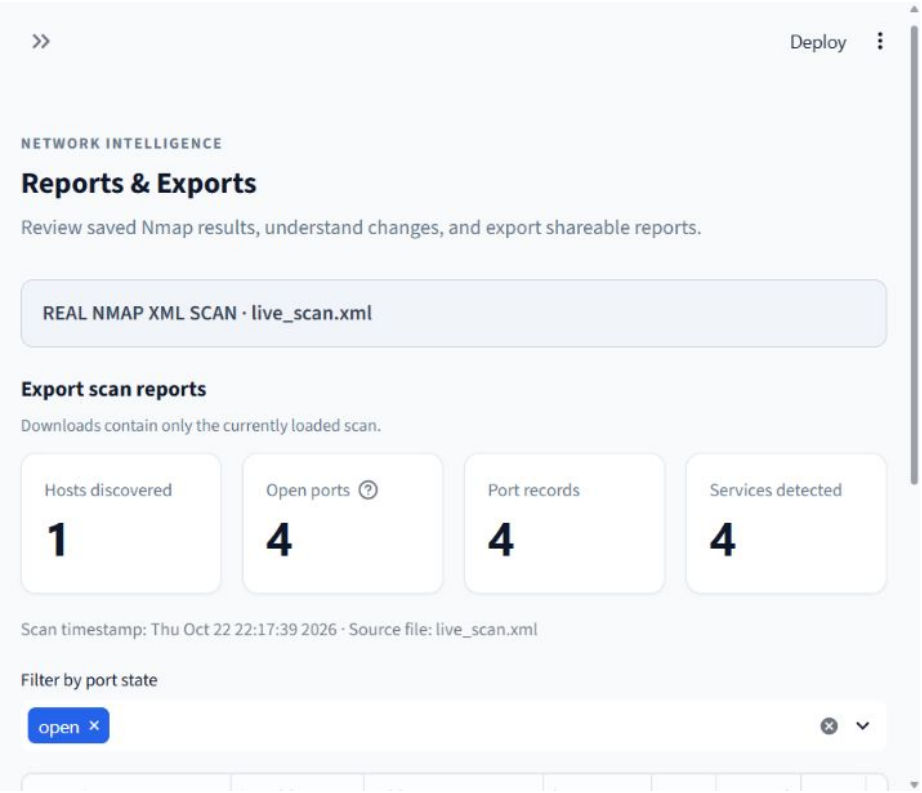


Fig. 6.6. Reports & Exports view for the active real scan.

NMAP COMMAND OUTPUT

6.8 Nmap record. The command screenshot shows the saved text output for the baseline scan, including Nmap command, target, four open TCP ports, 96 closed ports and one host up. XML is the machine-readable source consumed by NetMap. The image is an inspection of saved output, not evidence that the scan was rerun during report preparation.

Nmap service/version results are probe/fingerprint-based. The port 8080 product string "Oracle XML DB Enterprise Edition httpd" is Nmap's identification, not verified installed software [5].

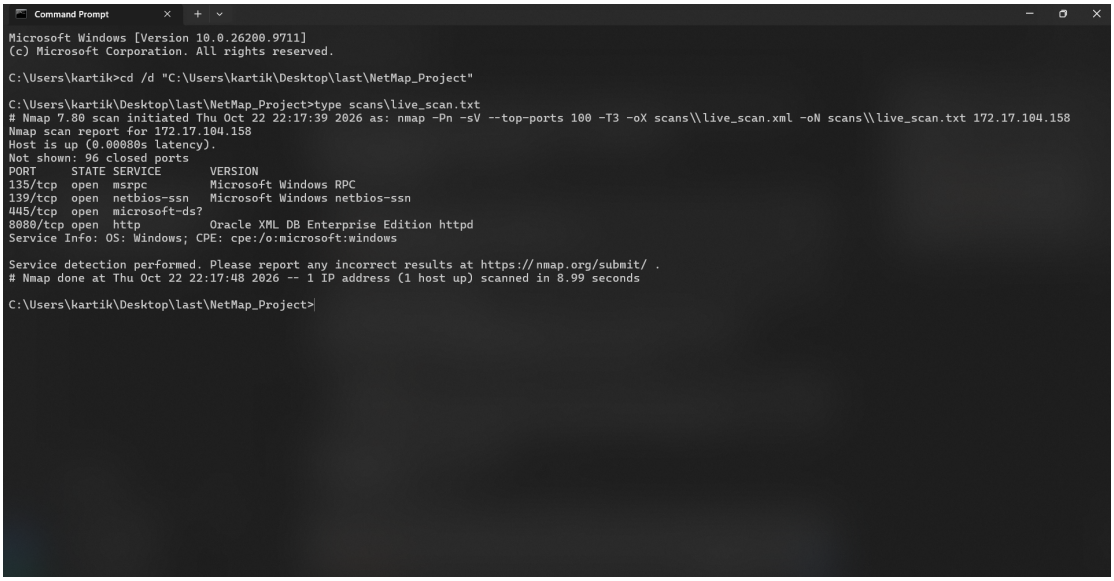


Fig. 6.7. Command Prompt displaying saved Nmap text output for the baseline scan.

WIRESHARK CAPTURE AND EVIDENCE LIMIT

6.9 Packet evidence. The supplied Wireshark window is open to `nmap_scan_capture.pcapng` and shows local loopback addresses 127.0.0.1. The recorded command-line inspection reports 94 packets over roughly 45 seconds. A target-specific SYN filter for 172.17.104.158 and ports 135, 139, 445 and 8080 returns no matching scan SYN frames. A broad target-address filter matches six ICMP host-unreachable messages quoting SYN attempts to unrelated hosts on port 7680.

This capture is not evidence that the scanned target accepted or rejected probes on the four reported ports. A new capture should start on the interface carrying authorized target traffic before repeating a scan; packet endpoints and display filter should be checked before claiming corroboration. A Wireshark display filter changes which packets are shown, not which packets remain stored [7].

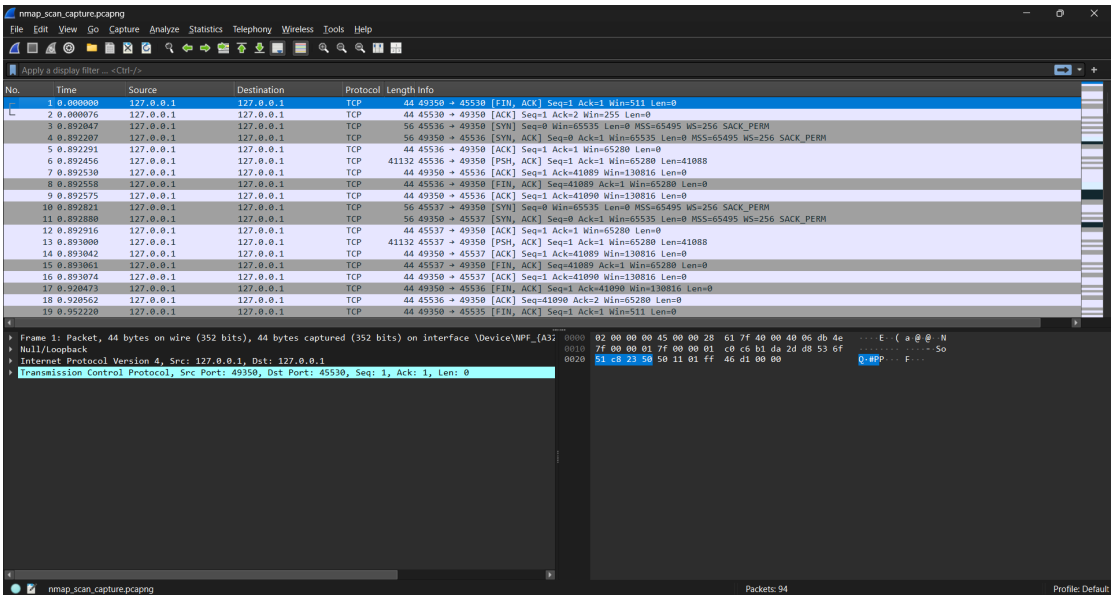


Fig. 6.8. Wireshark showing the saved capture; visible loopback traffic does not corroborate the target scan.

OBSERVED SCAN RESULTS

7.1 Experimental setup. Saved outputs identify Nmap 7.80 and target 172.17.104.158. Both commands use `-Pn -sV --top-ports 100 -T3`, producing XML and normal text output. Each scan is limited to one private address and a defined set of common TCP ports. The logs record elapsed times of 8.99 and 9.52 seconds.

The scans began at 22:17:39 and 22:39:55 on 22 October 2026. XML and text files are the primary evidence. The same target and options make the pair suitable for a focused snapshot comparison, although two observations cannot describe long-term availability or every service.

No firewall rule changes, service-stop experiment, virtual-machine experiment or new scan during this report update is claimed. Existing scan artefacts are preserved.

7.2 Summary. Both XML files record the same host status and the same four open TCP endpoints. Normal output reports 96 closed ports among the selected 100. Ports outside the scan scope were not assessed.

Nmap identifies 135 as msrpc and fingerprints Microsoft Windows RPC; 139 as netbios-ssn and fingerprints Microsoft Windows netbios-ssn; 445 as microsoft-ds with a table-derived service label; and 8080 as http with a product fingerprint. Port 445 has lower XML service confidence than the probed labels. An open port is an observation, not a vulnerability finding.

Table 7.1. Summary of saved Nmap scans

Scan file	Start time	Hosts up	Scope	Open TCP ports	Closed
live_scan.xml	22 Oct 2026 22:17:39	1	Top 100	135, 139, 445, 8080	96
live_scan_after.xml	22 Oct 2026 22:39:55	1	Top 100	135, 139, 445, 8080	96

COMPARISON RESULT AND INTERPRETATION

7.3 Before/after comparison. The comparator evaluates endpoint identity (172.17.104.158, port, tcp). The intersection of open keys contains all four endpoints in both scans. For each shared endpoint, the service label built from service name, product and version is unchanged. Thus the result has zero opened, zero no-longer-open and zero service-changed records.

The empty result means no difference under NetMap's defined rules for these files. It does not guarantee that all network traffic, closed/filtered states, application behaviour, host configuration or services outside the selected ports remained unchanged.

Saved filenames, timestamps, target and command options permit another reviewer to reproduce the comparison. Output metadata retains the scan identities so an empty CSV is not confused with a missing or failed comparison.

TESTS, COST AND LIMITATIONS

7.4 Verification. The recorded suite contains 15 passing tests. Python compilation passed for the explicit application/parser/test files, and `pip check` reported no broken requirements. The UI evidence covers real scan selection, Overview, Host Discovery, Ports & Services, Comparison and export controls. The stored report files were inspected locally; browser downloads were not re-opened during this report update.

7.5 Cost. The project uses open-source software and an existing Windows computer/network environment. No dedicated hardware purchase or paid software cost is documented. Student time, institutional infrastructure and licensing arrangements were not measured, so a precise monetary total is not asserted.

7.6 Limitations. The scan covers one address, TCP only and 100 common ports. Service detection can misidentify banners; the saved capture does not match target traffic. Tests partly use controlled fixtures and do not replace manual browser review. No vulnerability scan, exploit attempt, firewall experiment or production deployment was performed.

Table 7.2. Verification outcomes

Check	Recorded outcome	What it establishes
unittest discover -s tests -v	15 passed	Parser, comparison, dashboard and export cases
py_compile explicit files	Passed	Compilation for listed project modules
pip check	No broken requirements	Installed package metadata consistency
Streamlit / browser UI	Existing manual evidence	App states documented in screenshots
New scan / opened UI downloads	Not performed here	No current live-scan/download claim

CHAPTER 8 SUMMARY

NetMap demonstrates a traceable workflow for reviewing saved Nmap XML. It provides a scan-selection gate, source-labelled real-scan Overview, host and port/service views, a two-file comparison, and CSV/HTML reporting. A normalized data model keeps the active scan consistent across views, while the comparator matches open endpoints by IP, port and protocol and compares service labels.

The verified files show one host up and four open TCP ports (135, 139, 445 and 8080) on 172.17.104.158, with 96 closed ports among the top 100 scanned. Both scans use Nmap 7.80 and equivalent options. The comparison reports no differences. The recorded 15-test suite, compilation check and package consistency check passed.

The Wireshark evidence is separate and inconclusive for these scan results. The visible capture is loopback traffic; recorded target-specific inspection found no matching SYN frames for scanned endpoints. The correct conclusion is that the saved XML supports the reported port observations, while this capture does not independently confirm them.

Future work could add scan-history indexing, richer comparison filters, optional IPv6/UDP fixtures, accessible chart descriptions and a fresh aligned capture exercise. Active probing should remain explicit and authorized. NetMap is not a vulnerability scanner, and service fingerprints should be verified before operational decisions.

CHAPTER 9 REFERENCES

[1] Nmap Project, "XML Output (-oX)," Nmap Network Scanning. https://nmap.org/book/output-formats-xml-output.html (accessed 23 October 2026).

[2] Nmap Project, "Output," Nmap Reference Guide. https://nmap.org/book/man-output.html (accessed 23 October 2026).

[3] Python Software Foundation, "xml.etree.ElementTree - The ElementTree XML API," Python 3 Documentation. https://docs.python.org/3/library/xml.etree.elementtree.html (accessed 23 October 2026).

[4] Nmap Project, "Port Scanning Techniques," Nmap Network Scanning. https://nmap.org/book/man-port-scanning-techniques.html (accessed 23 October 2026).

[5] Nmap Project, "Service and Application Version Detection," Nmap Network Scanning. https://nmap.org/book/vscan.html (accessed 23 October 2026).

[6] Nmap Project, "Service and Version Detection," Nmap Reference Guide. https://nmap.org/book/man-version-detection.html (accessed 23 October 2026).

REFERENCES (CONTINUED)

[7] Wireshark Foundation, "Wireshark User's Guide, Section 6.3: Filtering Packets While Viewing." https://www.wireshark.org/docs/wsug_html_chunked/ChWorkDisplayFilterSection.html (accessed 23 October 2026).

[8] Streamlit, "Streamlit Documentation." https://docs.streamlit.io/ (accessed 23 October 2026).

[9] J. Postel, "Transmission Control Protocol," RFC 793, IETF, September 1981. https://www.rfc-editor.org/rfc/rfc793 (accessed 23 October 2026).

[10] Nmap Project, "Nmap Reference Guide." https://nmap.org/book/man.html (accessed 23 October 2026).

[11] Wireshark Foundation, "Wireshark User's Guide." https://www.wireshark.org/docs/wsug_html/ (accessed 23 October 2026).

[12] NetMap project artefacts: `netmap.py`, `dashboard.py`, `scans/live_scan.xml`, `scans/live_scan_after.xml`, corresponding text files, saved CSV/HTML outputs, and `evidence/verification_results.txt` (accessed 23 October 2026).

APPENDIX A REPRODUCTION COMMANDS AND ARTEFACTS

A.1 Launch the local dashboard from the project directory:

Select `scans/live_scan.xml` to inspect the baseline. In Comparison, choose `live_scan.xml` as Baseline and `live_scan_after.xml` as After. The dashboard is a review tool and does not issue Nmap commands.

A.2 Read the saved text record without modifying it:

A.3 Existing artefacts: scan XML/text are in scans/; PCAPNG is `scans/nmap_scan_capture.pcapng`; CSV/HTML are in outputs/; real dashboard screenshots and verification record are in evidence/. Preserve original scan and capture files when reproducing analysis.

Automated test command recorded: `py -m unittest discover -s tests -v` - 15 tests passed in 16.526 seconds. Compilation recorded: `py -m py_compile dashboard.py netmap.py tests/test_dashboard_overview.py tests/test_netmap.py` - exit code 0. Dependency metadata: `py -m pip check` - "No broken requirements found."

The original verification log also contains an initial compilation command using a nonexistent `src/` path. That attempt failed; the explicit corrected command above passed. A prior launcher run started Streamlit successfully and was stopped with Ctrl+C, so the interruption exit code 1 is not a clean shutdown test.

Interpretation checklist: keep scan scope and time attached to each finding; treat service strings as fingerprints; call an endpoint "no longer open" unless later evidence confirms closed; keep PCAPNG observations independent; do not infer vulnerabilities from open ports; and do not mix test fixtures with real scans.

```
py -m pip install -r requirements.txt
py -m streamlit run dashboard.py

type scans\live_scan.txt
type scans\live_scan_after.txt
```